SMART LIQUIDATION ENGINE – v1.0 (Refined)

1. DEFINITIONS / ENUMS

MarketState ∈ {RISK_ON, NEUTRAL, RISK_OFF}

asset_type_i ∈ {stock, MF, gold, FD, bond, real_estate, cash}

asset_class_i ∈ {equity, debt, gold, real_estate, cash}

2. USER INPUT

R = required liquidation amount (currency)

TU = time urgency (days, integer)

3. SYSTEM INPUT (AUTO, PER ASSET i)

For each asset i:

asset_id_i

asset_type_i

asset_class_i

timestamp

$V_i$ = current value

$P_i$ = purchase value

$L_i$ ∈ = liquidity score

$\mu_i$ = expected return (future, annualized or consistent horizon)

$\sigma_i$ = volatility

$DD_i$ = max drawdown

$LP_i$ ∈ = loss probability (ML)

$T_i$ = tax rate (effective rate on realized gain)

$LC_i$ ∈ {0,1} = lock-in flag (1 = locked, cannot sell)

$MC_i$ ∈ {0,1} = maturity flag (1 = not yet matured, cannot sell)

$SA_i$ ∈ {0,1} = sell allowed flag (1 = allowed, 0 = regulatory/other block)

4. PORTFOLIO VARIABLES

TotalPortfolio = $\Sigma_i V_i$

$W_i = V_i$ / TotalPortfolio

(Optional) $TargetWeight_i$ (per asset or per asset_class)

$previous_score_i$ = last stored $AdjSellScore_i$ (if not available, set to current $AdjSellScore_i$ once computed)

5. DERIVED VARIABLES

For each asset i:

$G_i = V_i - P_i$

$Tax_i = \max(0, G_i) \times T_i$ (no tax credit for losses by default)

$OC_i = \mu_i \times V_i$ (opportunity cost of selling – absolute expected value of holding)

$AdjReturn_i = \mu_i \times (1 - LP_i)$ (risk-adjusted expected return)

$Risk_i = 0.5 \times \sigma_i + 0.5 \times DD_i$

6. NORMALIZATION

For any feature x over eligible assets S (defined later):

If $\max(x_S) = \min(x_S)$:

$norm(x_i) = 0.5$ (neutral)

Else:

$norm(x_i) = (x_i - \min(x_S)) / (\max(x_S) - \min(x_S))$

Compute:

$L_{i_norm}$

$Risk_{i_norm}$

$Tax_{i_norm}$

$AdjReturn_{i_norm}$

$LP_{i_norm}$

$OC_{i_norm}$

Missing values for any feature → 0.5 (neutral).

7. BASE WEIGHTS

$w_L = 0.25$ (Liquidity)

$w_R = 0.20$ (Risk)

$w_T = 0.15$ (Tax)

$w_P = 0.15$ (Loss probability)

$w_{OC} = 0.15$ (Opportunity cost)

$w_E = 0.10$ (Adjusted return, negative in score)

8. MARKET ADAPTATION

If MarketState = RISK_OFF:

$w_L = 0.30$

$w_R = 0.30$

$w_P = 0.20$

$w_E = 0.05$

If MarketState = RISK_ON:

w_L = 0.20

w_R = 0.15

w_P = 0.10

w_E = 0.25

If MarketState = NEUTRAL:

Keep base weights.

9. URGENCY ADJUSTMENT

If TU ≤ 7 (high urgency):

w_L += 0.10

w_E −= 0.05

Then re-normalize all weights to sum to 1:

SumW = w_L + w_R + w_T + w_P + w_OC + w_E

For each weight w_k: w_k = w_k / SumW

10. FILTERING – ELIGIBLE SET S

Define the eligible set:

S = { i | LC_i = 0 AND MC_i = 0 AND SA_i = 1 }

If S is empty:

Return: HOLD (no eligible assets to liquidate).

All scoring and allocation below operate on S.

11. CORE SELL SCORE

For each i ∈ S:

SellScore_i =

( w_L × L_i_norm )

( w_R × Risk_i_norm )

( w_T × Tax_i_norm )

( w_P × LP_i_norm )

( w_OC × OC_i_norm )

− ( w_E × AdjReturn_i_norm )

Higher SellScore_i → more attractive to sell.

12. INTELLIGENCE LAYER (CONTEXT ADJUSTMENTS, WITH CAP)

Initialize:

AdjSellScore_i = SellScore_i

Apply small context-based adjustments:

Urgency – favor liquid assets when urgent:

If TU ≤ 7:

AdjSellScore_i += 0.05 × L_i_norm

High loss probability – penalize risky assets:

If LP_i > 0.6:

AdjSellScore_i += 0.05 × LP_i_norm

Real estate philosophy (choose one and document in implementation): Option A – Avoid forced selling of illiquid real estate (default)

If asset_class_i = real_estate:

AdjSellScore_i −= 0.05 × (1 − L_i_norm)

(Illiquid real estate is slightly protected; you sell it only if really necessary.)

Option B – Use real estate as “one big ticket” when needed

If asset_class_i = real_estate:

AdjSellScore_i += 0.05 × (1 − L_i_norm)

(Illiquid real estate becomes more likely to be sold in one shot.)

Choose ONE option for production and keep it consistent.

Risk-off regime – de-risk:

If MarketState = RISK_OFF:

AdjSellScore_i += 0.05 × Risk_i_norm

12A. CAP ON INTELLIGENCE ADJUSTMENT

To keep behavior stable, cap total adjustment from this layer per asset:

Define Δ_i = AdjSellScore_i − SellScore_i

If Δ_i > +0.15: set AdjSellScore_i = SellScore_i + 0.15

If Δ_i < −0.15: set AdjSellScore_i = SellScore_i − 0.15

(Exact ±0.15 is tunable; this prevents the intelligence layer from overpowering the core economic score.)

13. SCORE STABILITY CHECK

To avoid reacting to noise:

If |AdjSellScore_i − previous_score_i| < 0.10:

Mark asset i as STABLE.

STABLE assets are lower priority for selling and will be used only if high-score non-STABLE assets cannot satisfy R.

Implementation hint:

In the sort step, treat STABLE assets as a second-tier bucket (same sorting keys but used after non-STABLE assets if still Remaining > 0).

14. DYNAMIC SELL FRACTION (α_i)

Define sell fraction α_i based on AdjSellScore_i:

If AdjSellScore_i ≥ 0.70 → α_i = 0.70

If 0.40 ≤ AdjSellScore_i < 0.70 → α_i = 0.40

If AdjSellScore_i < 0.40 → α_i = 0.20

These bands are tunable, but fixed here for v1.0.

15. LIQUIDATION ENGINE – MULTI-ASSET PLAN

Initialize:

Remaining = R

15.1 Sort order

Create ordered list of eligible assets S_sorted as:

Non-STABLE assets first, then STABLE assets.

Within each group, sort by:

AdjSellScore_i (desc)

L_i_norm (desc)

Tax_i_norm (asc)

15.2 Allocation loop

For each asset i in S_sorted:

If Remaining ≤ 0: break

SellAmount_i = min(V_i × α_i, Remaining)

Remaining −= SellAmount_i

If, after this loop, Remaining > 0:

The system can mark shortfall and optionally:

Re-run with relaxed constraints, or

Flag to advisor/user for manual action.

16. SAFETY LAYER (POST-ALLOCATION)

16.1 Asset class concentration cap

Compute:

TotalSell = Σ_i SellAmount_i

For each asset_class c:

ClassSell_c = Σ_{i in class c} SellAmount_i

If ClassSell_c / TotalSell > 0.80:

Cap ClassSell_c at 0.80 × TotalSell.

Reduce individual SellAmount_i in class c proportionally so that:

Σ_{i in class c} SellAmount_i_new = 0.80 × TotalSell.

This frees some “excess” sell amount to reallocate to other asset classes.

16.2 Redistribution logic

Take the freed amount (ExcessSell) and redistribute it to assets:

Build a new list of candidate assets:

Eligible assets not in any already-capped asset_class.

Sorted by the same order as before (AdjSellScore_i desc, L_i_norm desc, Tax_i_norm asc).

Loop through candidates and increase SellAmount_i up to (V_i × α_i) while ExcessSell > 0:

Addable_i = max(0, V_i × α_i − SellAmount_i_current)

Δ = min(Addable_i, ExcessSell)

SellAmount_i_new = SellAmount_i_current + Δ

ExcessSell −= Δ

16.3 Minimum diversification

After safety layer:

Ensure at least 2 asset_class have Σ V_i_after_sell > 0.

If only 1 asset_class would remain non-zero, reduce the last class’s SellAmount_i just enough to keep at least 2 classes alive (simple portfolio survival rule).

17. SINGLE ASSET OPTIMIZATION (OVERRIDE CHECK)

Check if any single asset can meet R:

If ∃ i with V_i ≥ R: Compute:

SingleScore_i =

( w_L × L_i_norm )

( w_T × Tax_i_norm )

( w_R × Risk_i_norm )

− ( w_E × AdjReturn_i_norm )

Let i* = argmin_i SingleScore_i over assets with V_i ≥ R.

17.1 Override rule

Define:

MultiCost = weighted average SellScore over assets selected in multi-asset plan, weighted by SellAmount_i.

SingleCost = SingleScore_{i*}.

If:

SingleCost + δ < MultiCost (δ is a small positive margin, e.g. 0.05)

then:

Override multi-asset plan and instead:

Set SellAmount_{i*} = R

Set SellAmount_j = 0 for all j ≠ i*

If not:

Keep multi-asset plan.

(δ avoids flipping on tiny differences.)

18. CHURN CONTROL

Define churn control parameters:

MinHoldingPeriod = 14 days

CooldownAfterSell = 7 days

MinScoreChange = 0.10

Rules:

If holding period of asset i < MinHoldingPeriod → force HOLD (SellAmount_i = 0).

If last sell in asset i was less than CooldownAfterSell days ago → force HOLD.

If |AdjSellScore_i − previous_score_i| < MinScoreChange → treat as STABLE and deprioritize as in Section 13.

These rules apply before final SellAmount_i is confirmed.

19. REBALANCING TRIGGER (OPTIONAL)

If TargetWeight_i is provided for some assets or asset_class:

If |W_i − TargetWeight_i| > RebalanceThreshold (e.g., 5–10 percentage points):

Allow liquidation for rebalance even if R is small or zero.

This can be run as a separate mode: “Rebalance” vs “Raise Cash”.

20. CONFIDENCE SCORE

For each asset i:

raw_conf_i = 1 − variance(AdjReturn_i, LP_i, Risk_i)

Confidence_i = max(0, min(1, raw_conf_i))

Interpretation: more internally consistent risk/return signals → higher confidence.

(Exact formula can be refined later with ML; this version is a simple placeholder.)

21. OUTPUT (SORTED BY EXECUTION)

For each asset i with SellAmount_i > 0, output:

asset_id_i

asset_class_i

SellAmount_i

SellPercentage_i = SellAmount_i / V_i

Action_i ∈ {SELL, PARTIAL, HOLD} (based on SellPercentage_i bands, e.g. ≥ 0.7 = SELL, 0 < < 0.7 = PARTIAL, 0 = HOLD)

AdjSellScore_i

Confidence_i

Reason_i (human-readable explanation from explanation engine below)

21A. EXPLANATION ENGINE

For each asset i:

Compute factor contributions:

Liquidity contribution = w_L × L_i_norm

Risk contribution = w_R × Risk_i_norm

Tax contribution = w_T × Tax_i_norm

LP contribution = w_P × LP_i_norm

OC contribution = w_OC × OC_i_norm

AdjReturn contribution = − w_E × AdjReturn_i_norm

Then:

Take absolute values of contributions to rank importance.

Pick top 2–3 contributors with largest absolute contribution, ignoring any with magnitude < ε (small threshold like 0.02).

Generate explanation text, e.g.:

“Sell ₹X from [asset] because:

High liquidity

Higher risk relative to others

Lower adjusted return vs alternatives”

Ensure no contradictory or trivial factors are mentioned.

22. SYSTEM FLOW

End-to-end:

Data → ML signals → MarketState → Derived variables → Normalization → Base SellScore → Intelligence Layer (+ cap) → Stability & Churn checks → Multi-asset allocation → Safety layer

→ Single-asset override decision → Final SellAmounts → Explanation engine → Output.

23. CORE PRINCIPLE

The engine encodes the principle:

Decision =

Future Potential (AdjReturn)

− Risk

− Liquidity cost

− Tax impact

− Constraints (lock-ins, churn rules)

± Market condition

± Urgency

All parameters are tunable, but the structure is fixed and implementation-ready.

cna you give me ipdts pdf format , dont remove any ord , any word, kepe everythign as it is , each terma dn word i impratn